# Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



# Data Structures and Algorithms

## Graphs and Breadth-first Search

Prof. Reza Skandari

# Today

1. **Representing networks as graphs**
2. Graph search

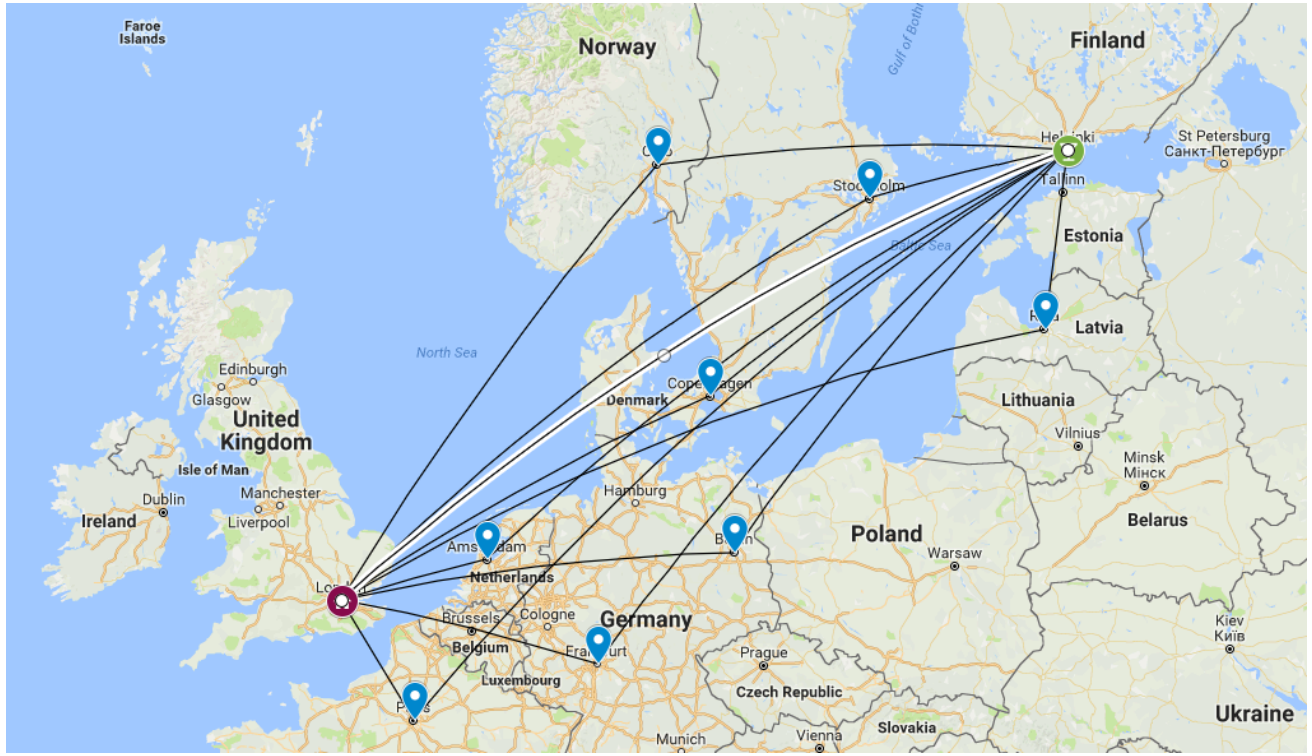
**This class brings the course's themes together**:

- **Data structures**: efficient ways to organize data, like a graph
- **OOP**: how we implement data structures, bundling data with the algorithms that operate on it
- **Algorithms**: e.g. breadth-first search, operating on that structure
- **Complexity analysis**: how we evaluate and compare algorithms ($O$)
- **Applications**: the real-world problems these ideas solve

# Graph problems

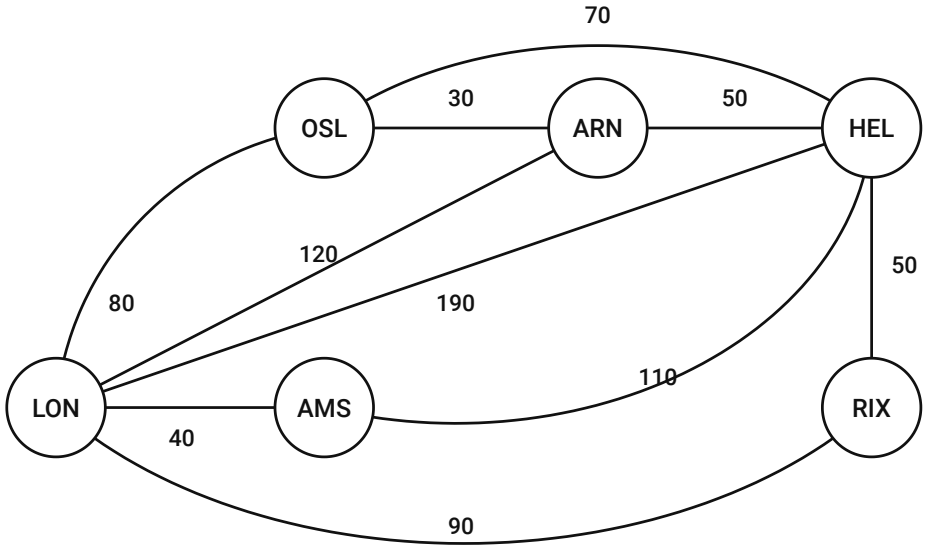
**Suppose you're travelling around Europe**

- Know **flights** and their **prices**
- **Goal**: fewest stops between cities, cheapest price, ...?



# Defining graphs

A set of **nodes** (or vertices) connected by **edges**.




Edges may or may not be directed → **directed** or **undirected** graph (Twitter vs Facebook?)

Edges may have **weights** (e.g. prices): weighted graph

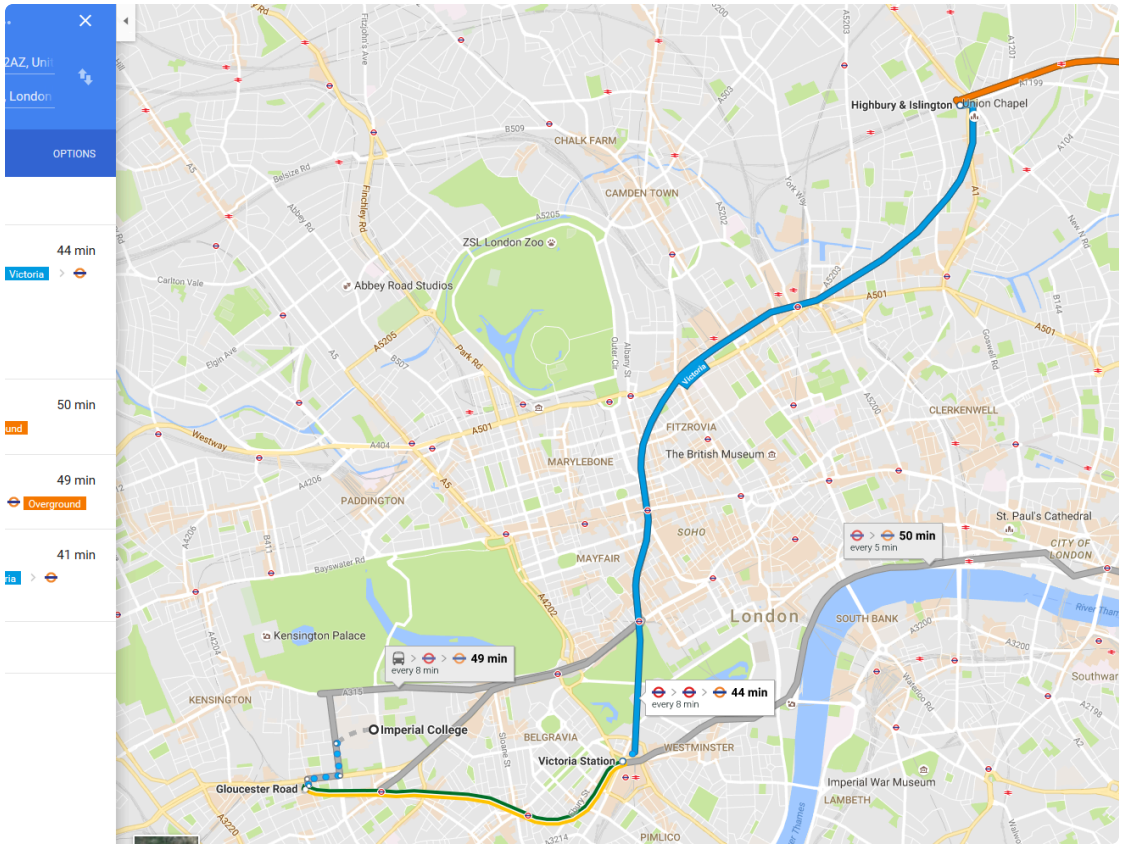
# Defining graphs: as a table of edges

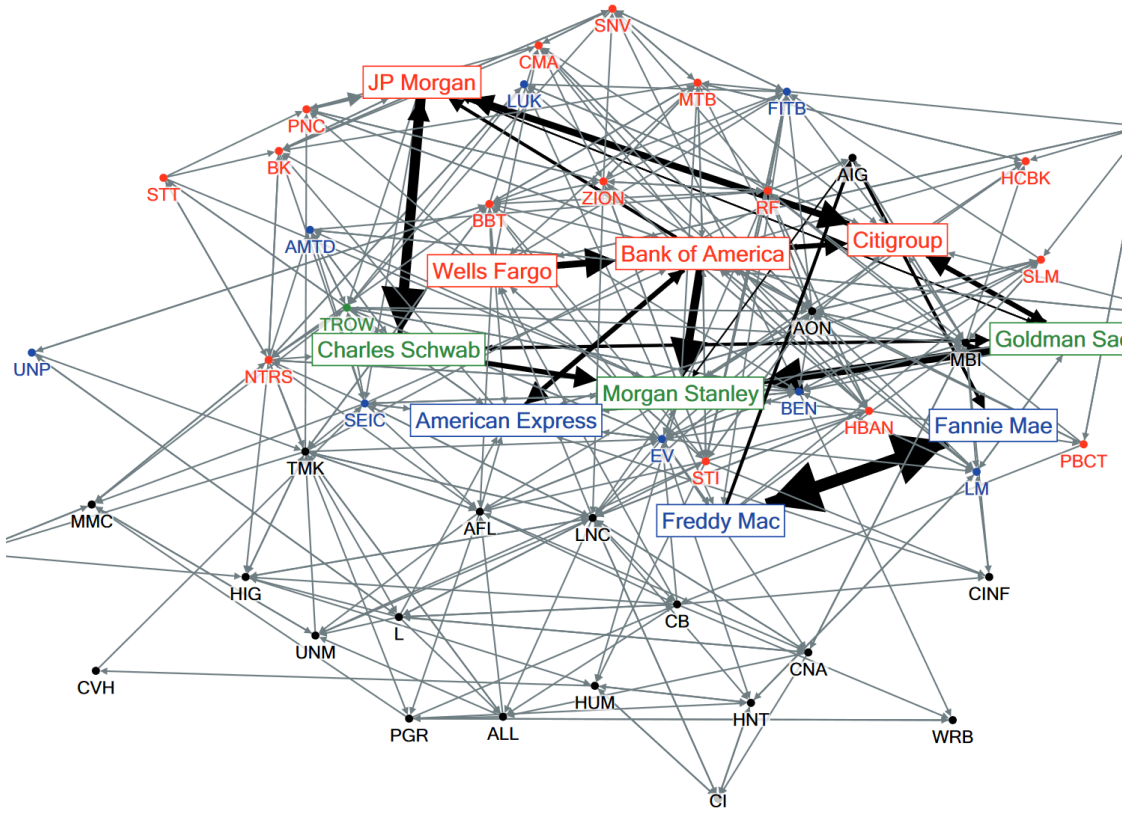
The same graph, listed as edges and weights:

| Edge | Weight |
|---|---|
| HEL-RIX | 50 |
| HEL-ARN | 50 |
| HEL-OSL | 70 |
| HEL-AMS | 110 |
| HEL-LON | 190 |

| Edge | Weight |
|---|---|
| RIX-LON | 90 |
| OSL-LON | 80 |
| ARN-LON | 120 |
| ARN-OSL | 30 |
| AMS-LON | 40 |

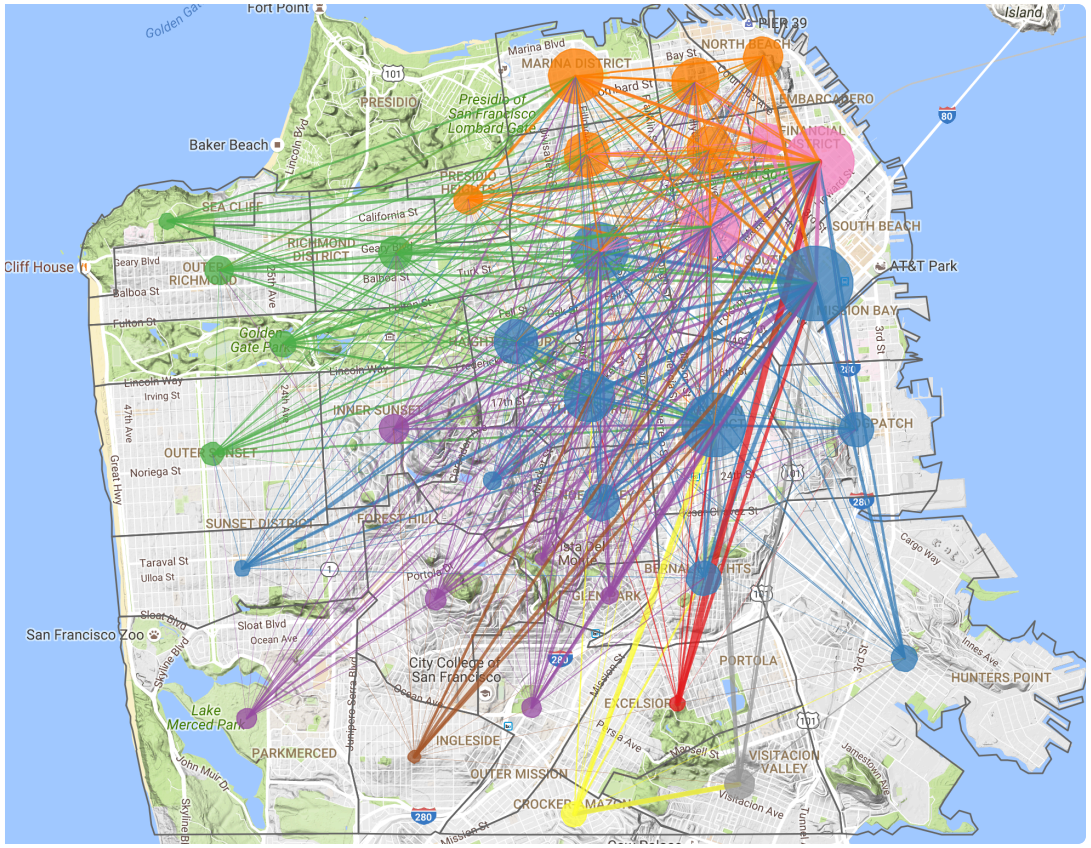
# Graphs are everywhere

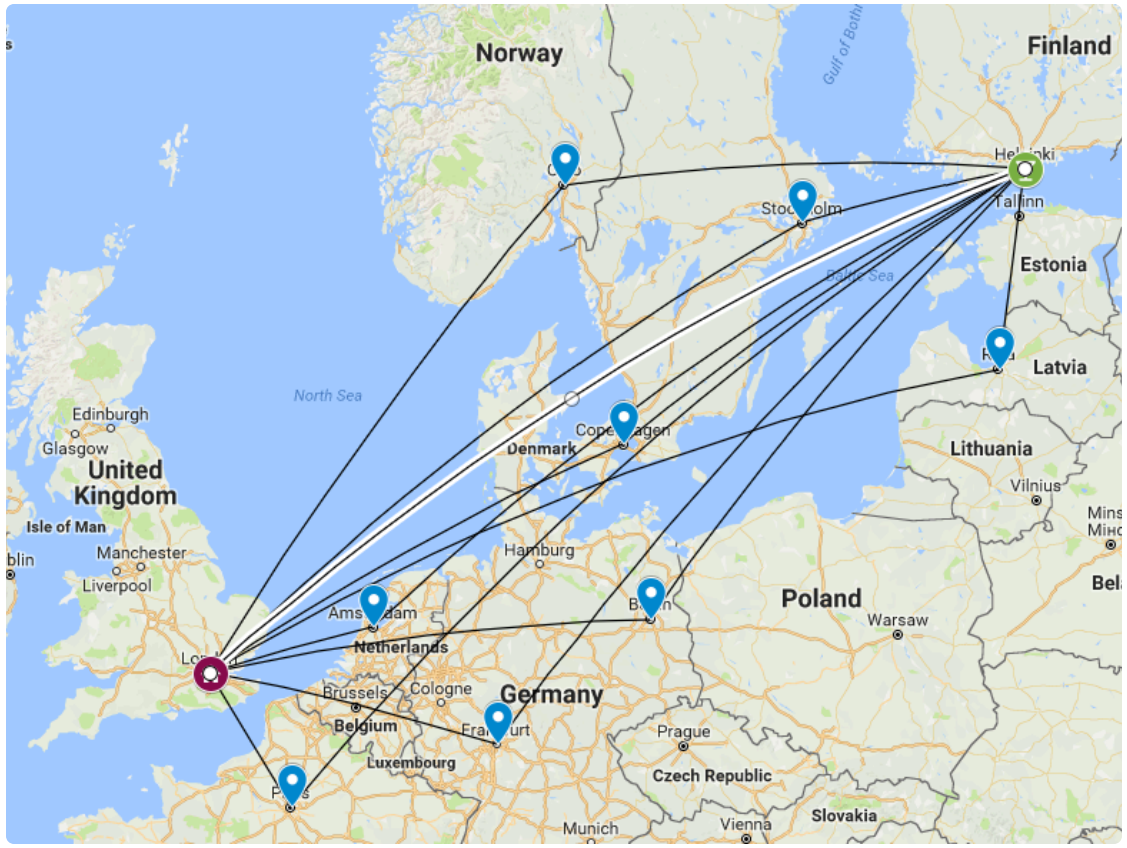



*Pics: Google Maps, Hautsch et al.*

# Graphs are everywhere




*Pic: Uber*

# Graphs are everywhere

Represent situations with **interesting pairwise relationships**

**Many applications**, some not obvious:

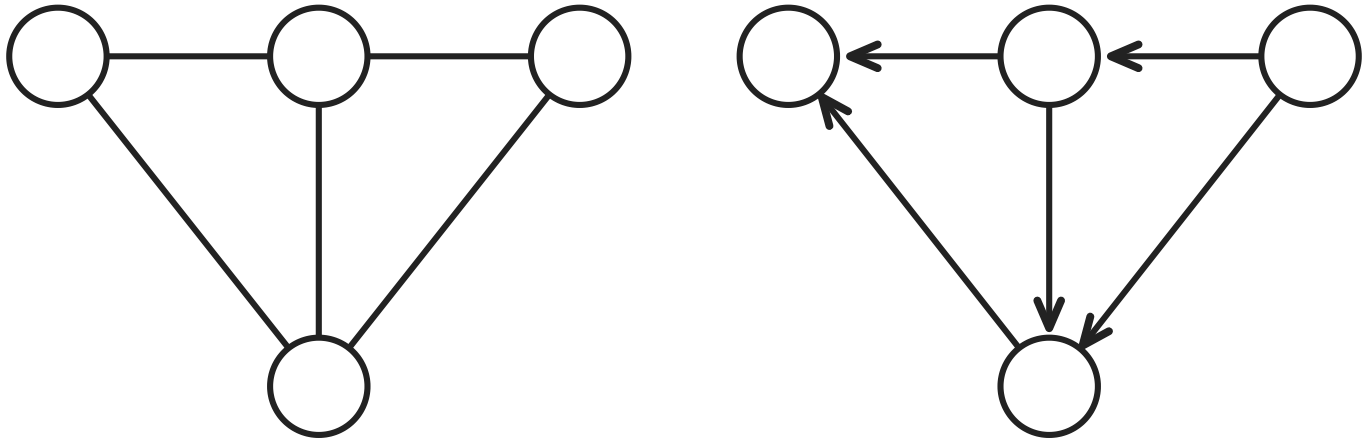
- **Physical networks**: roads, the web
- **Relationships**: social networks
- **Finance**: financial markets
- **Planning**: complex projects (dependencies between tasks)
- **Games**: chess (states of the board between player moves)

# Graph: collection of vertices and edges

Graph $G = (V, E)$: $V$ = set of $n$ **vertices**, $E$ = set of $m$ **edges**

- Edges can be **directed** or **undirected**
- **Connected** graph: there is a **path** between any two vertices

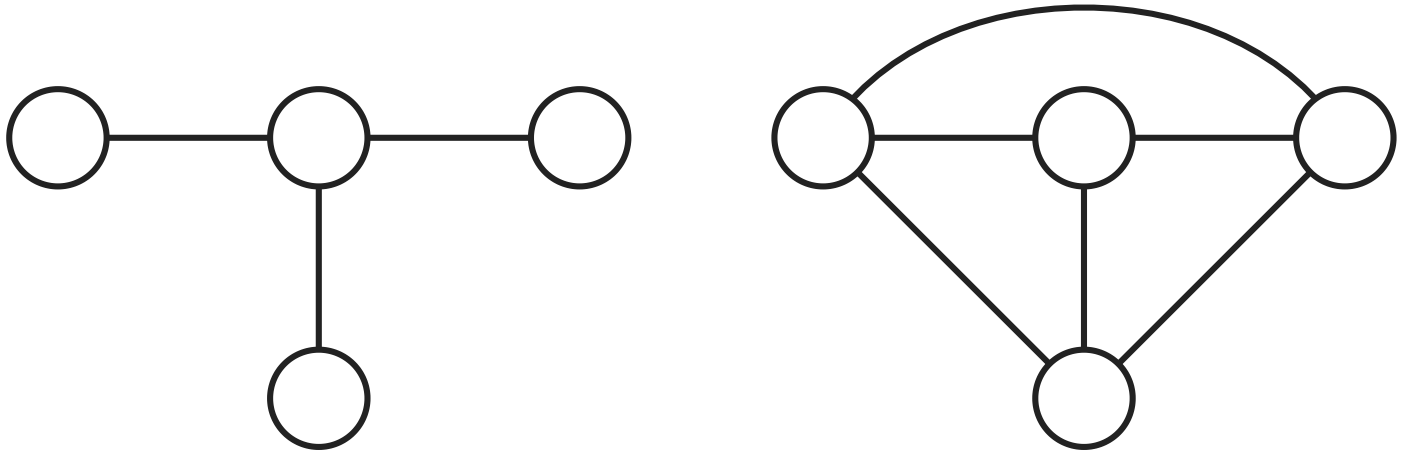
# Graph: collection of vertices and edges (continued)



**Which is connected?**

# Graph: collection of vertices and edges (continued)

**How many edges** can there be in a connected graph with $n$ vertices?



**Sparse** (left, $n-1$ edges) vs. **dense** (right, $n(n-1)/2$ edges) graphs

- Is Facebook dense or sparse?

# Graph: adjacency matrix

**Adjacency matrix**: size $n \times n$, where $A_{vw} = 1$ if there is an edge between vertices $v, w$ (unweighted): $n^2$ space required

| | LON | OSL | ARN | HEL | AMS | RIX |
|---|---|---|---|---|---|---|
| LON | 0 | 1 | 1 | 1 | 1 | 1 |
| OSL | 1 | 0 | 1 | 1 | 0 | 0 |
| ARN | 1 | 1 | 0 | 1 | 0 | 0 |
| HEL | 1 | 1 | 1 | 0 | 1 | 1 |
| AMS | 1 | 0 | 0 | 1 | 0 | 0 |
| RIX | 1 | 0 | 0 | 1 | 0 | 0 |

# Graph: adjacency lists

**Adjacency list**: connectivity represented using lists of edges and vertices

- Linear space requirement in $n$ (vertices), $m$ (edges)
- In practice: a **dictionary in Python**

```
graph = {
    'LON': {'OSL', 'ARN', 'HEL', 'AMS', 'RIX'},
    'OSL': {'LON', 'ARN', 'HEL'},
    'ARN': {'LON', 'OSL', 'HEL'},
    'HEL': {'LON', 'OSL', 'ARN', 'AMS', 'RIX'},
    'AMS': {'LON', 'HEL'},
    'RIX': {'LON', 'HEL'},
}
```

**Adjacency matrices vs lists**: dense vs sparse graphs: we'll be using adjacency lists

# Graph data structure

Class `Digraph` (directed graph) methods:

- `__init__`: creates a new (empty) `Digraph`
- `add_node(node)`: adds a node
- `add_edge(edge)`: adds an edge
- `children_of(node)`: returns all edges from node
- `has_node(node)`: returns `True` if it is in the graph, `False` otherwise
- `__str__`: returns a string with all the edges of the graph

Class `Graph(Digraph)`, an undirected graph:

- `add_edge(edge)`: adds an edge and its reverse edge to the graph
- Example of **inheritance**: `Graph` does everything that `Digraph` does, except the `add_edge` method is different

# Today

1. Representing networks as graphs
2. **Graph search**

# Searching a graph



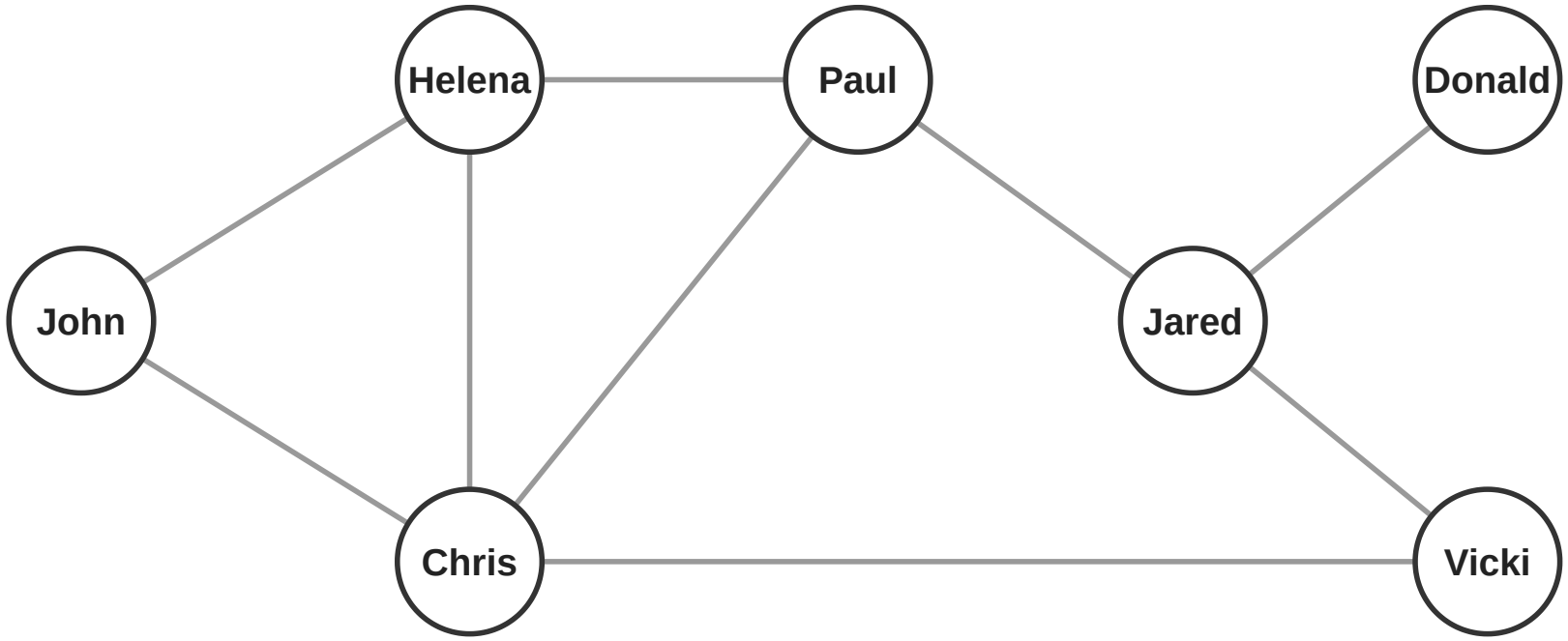

**Small worlds** phenomenon:

- Six degrees of separation (Marconi, Milgram, Guare)
- Hollywood: Kevin Bacon numbers

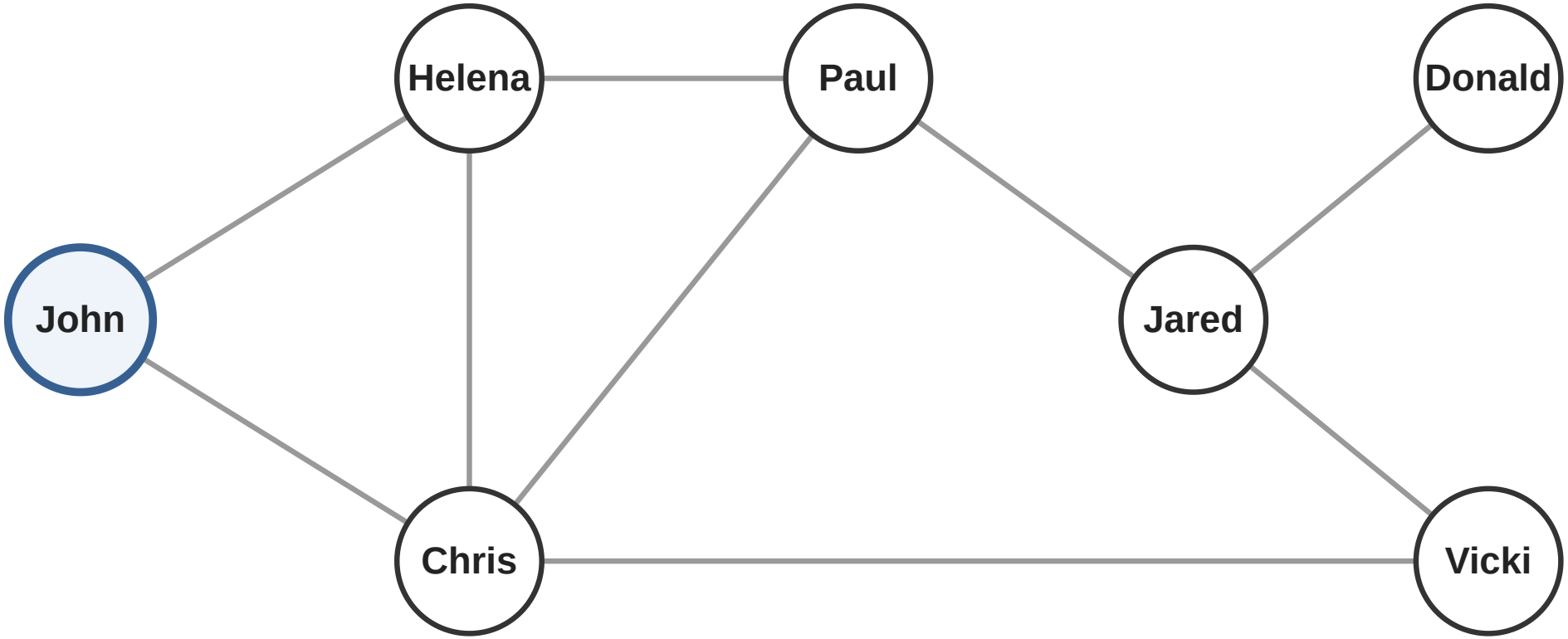
# Example: social network

How to find the fewest steps to someone on a social network?

Idea: Me? My friends? Friends' friends? Friends' friends' friends? ...

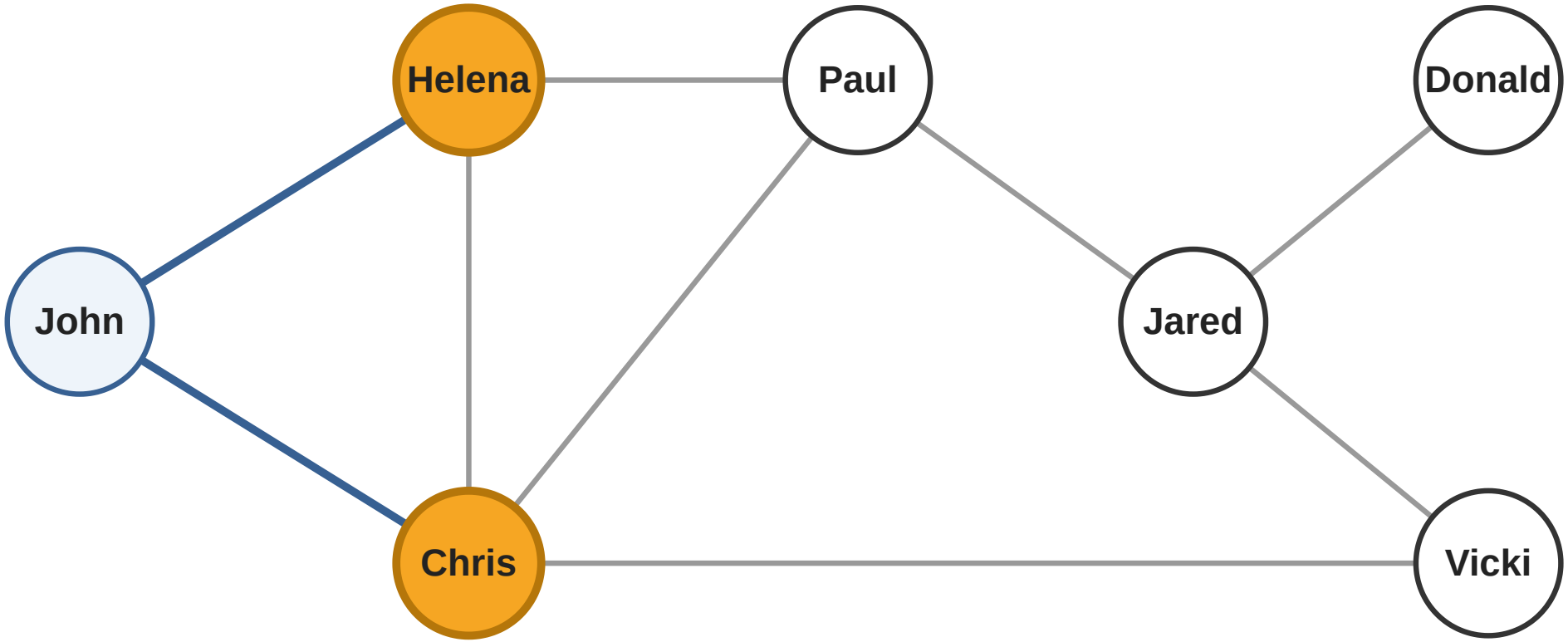


# Example: social network (me)



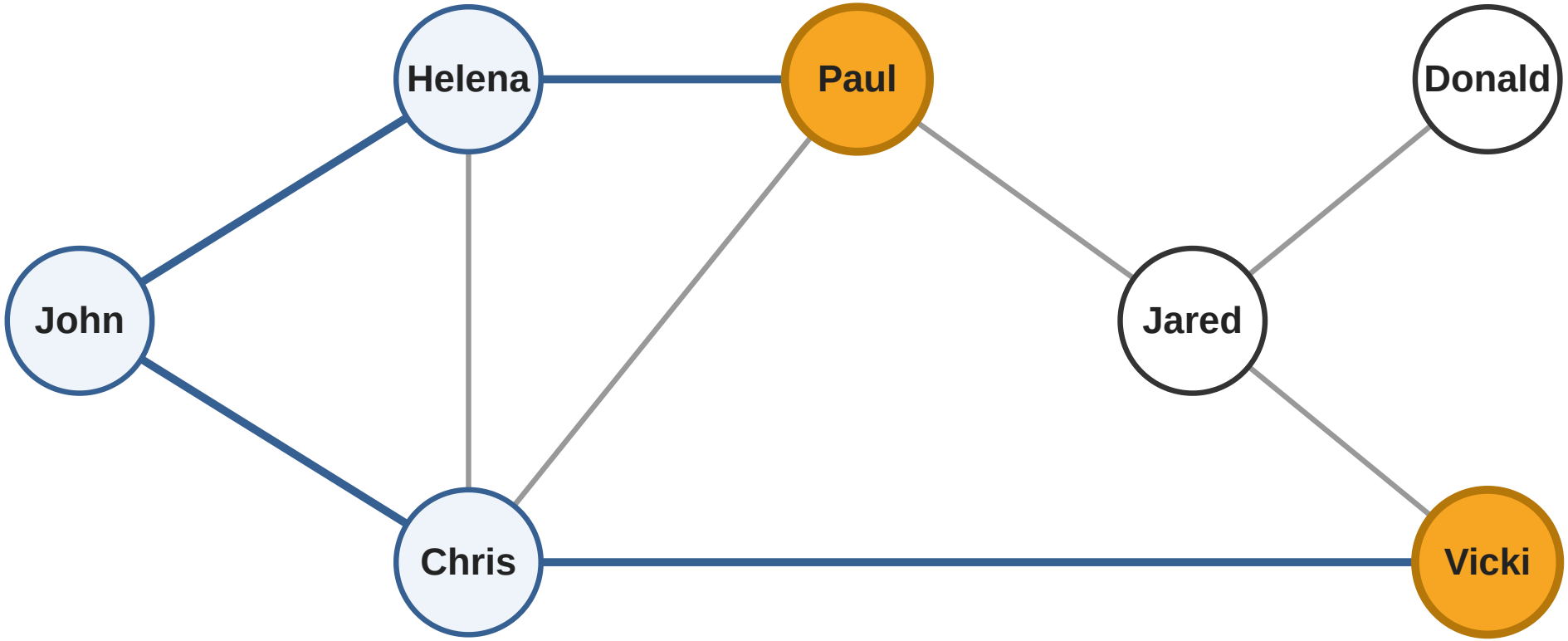

Start from **John**.

# Example: social network (my friends)



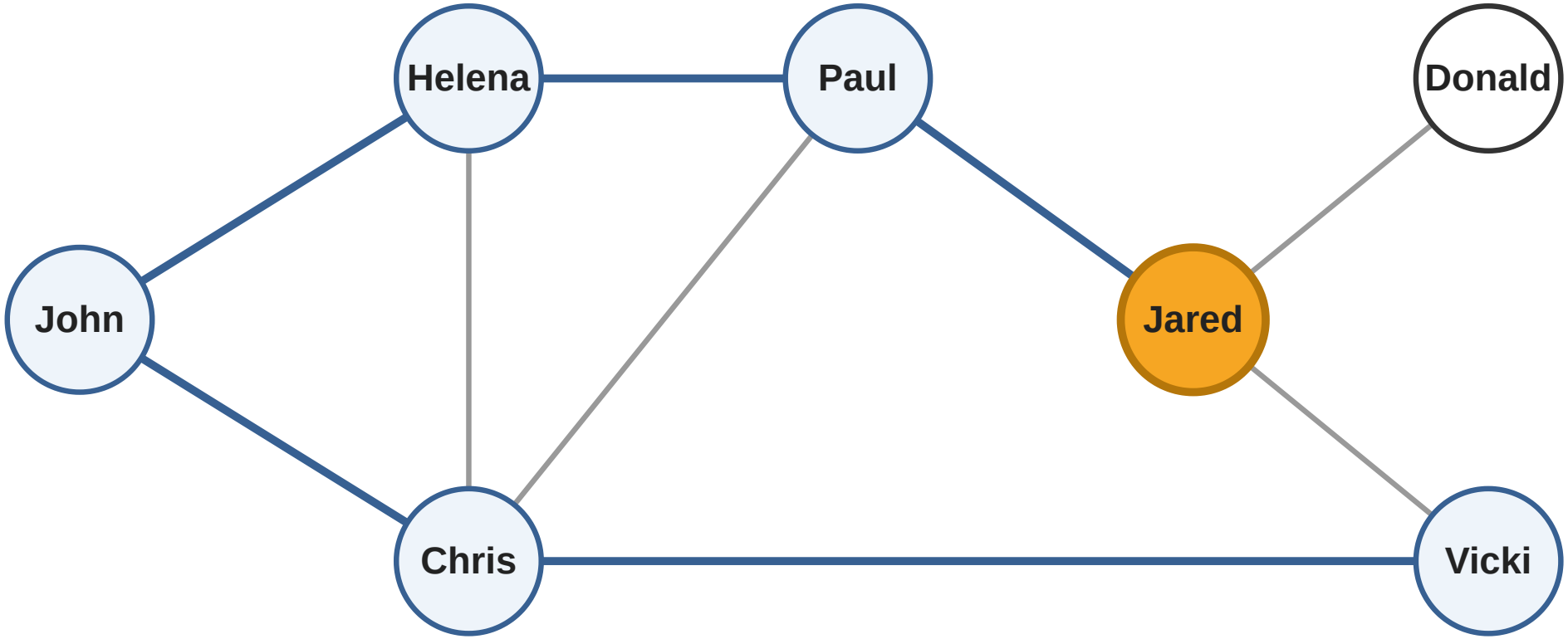

John's friends: **Helena** and **Chris**.

# Example: social network (friends' friends)



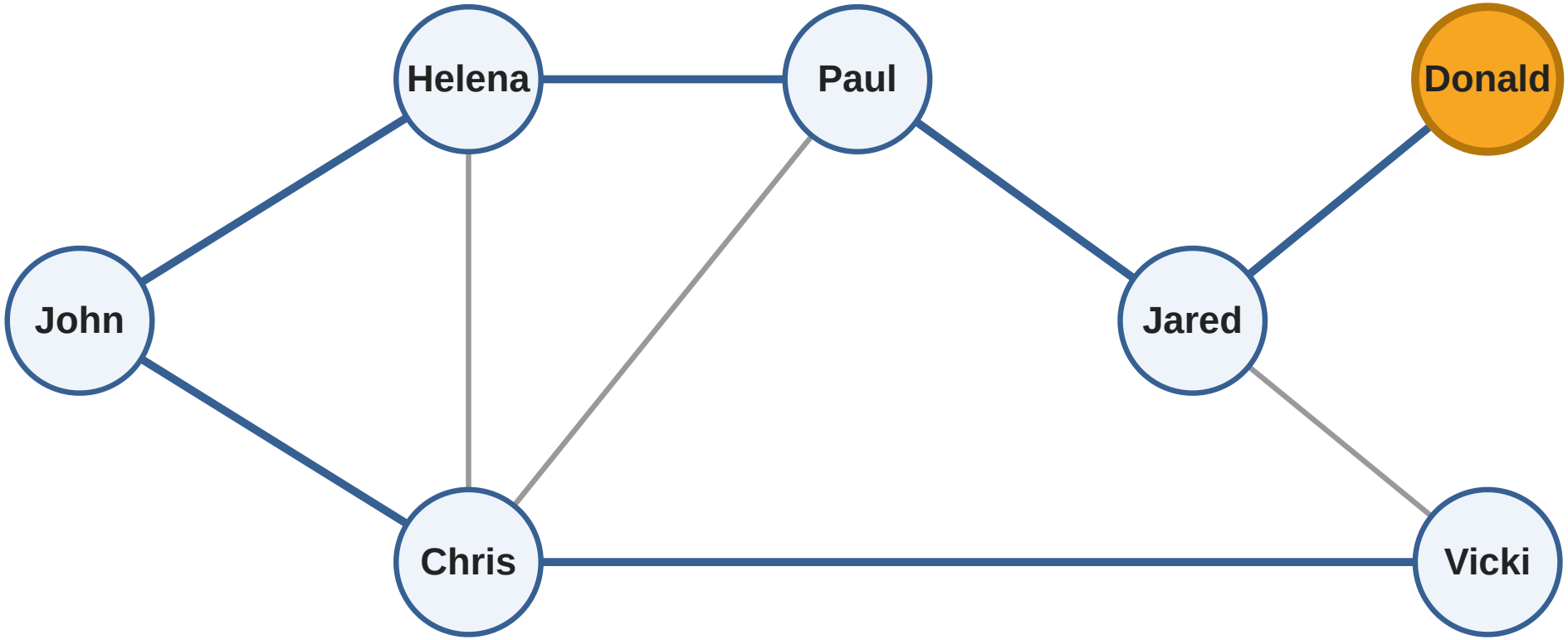

Helena and Chris's (new) friends: **Paul** and **Vicki**.

# Example: social network (one more layer)




Paul's (new) friend: **Jared**.

# Example: social network (everyone reached)




Jared's (new) friend: **Donald**. Everyone is now reached.

# Breadth-first search (BFS)

**Idea**: Explore nodes in layers

**Plan**:

1. How to explore a graph in layers?
2. How much work is it to go through the entire graph?
3. How to get shortest distances and paths?

# Exploring in layers: BFS algorithm

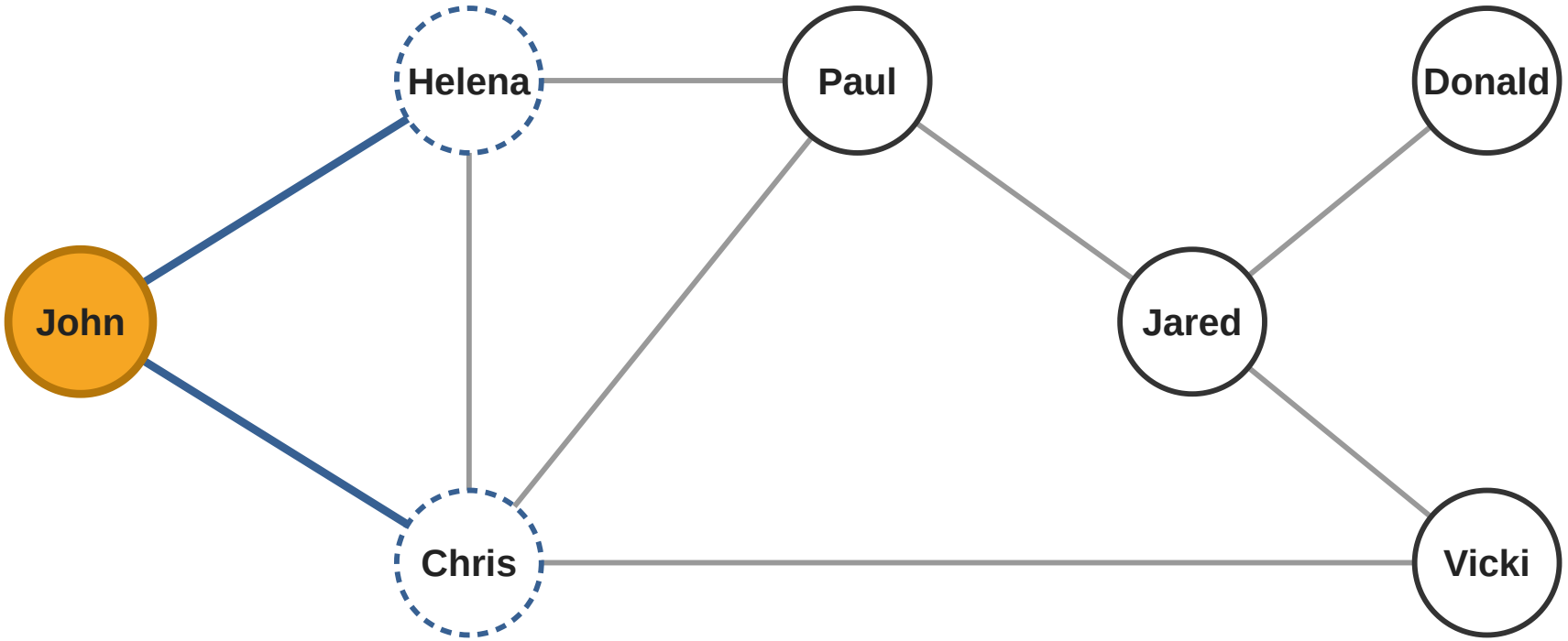
**Algorithm**: BFS(graph $G$, start node $s$)

- **Initialise**: mark all nodes unexplored except $s$ explored
- Use $Q$ = queue data structure; add $s$ to $Q$
- **Main loop**: While $Q$ is not empty:
  - Remove the first node of $Q$ and call it $v$
  - For each edge $(v, w)$: if $w$ unexplored, mark $w$ explored and add $w$ to $Q$

What is a **queue**?

- First in, first out (just like in a cafe)
- Remove from front, insert to back in constant time $O(1)$
- Here we keep a queue of nodes to visit next

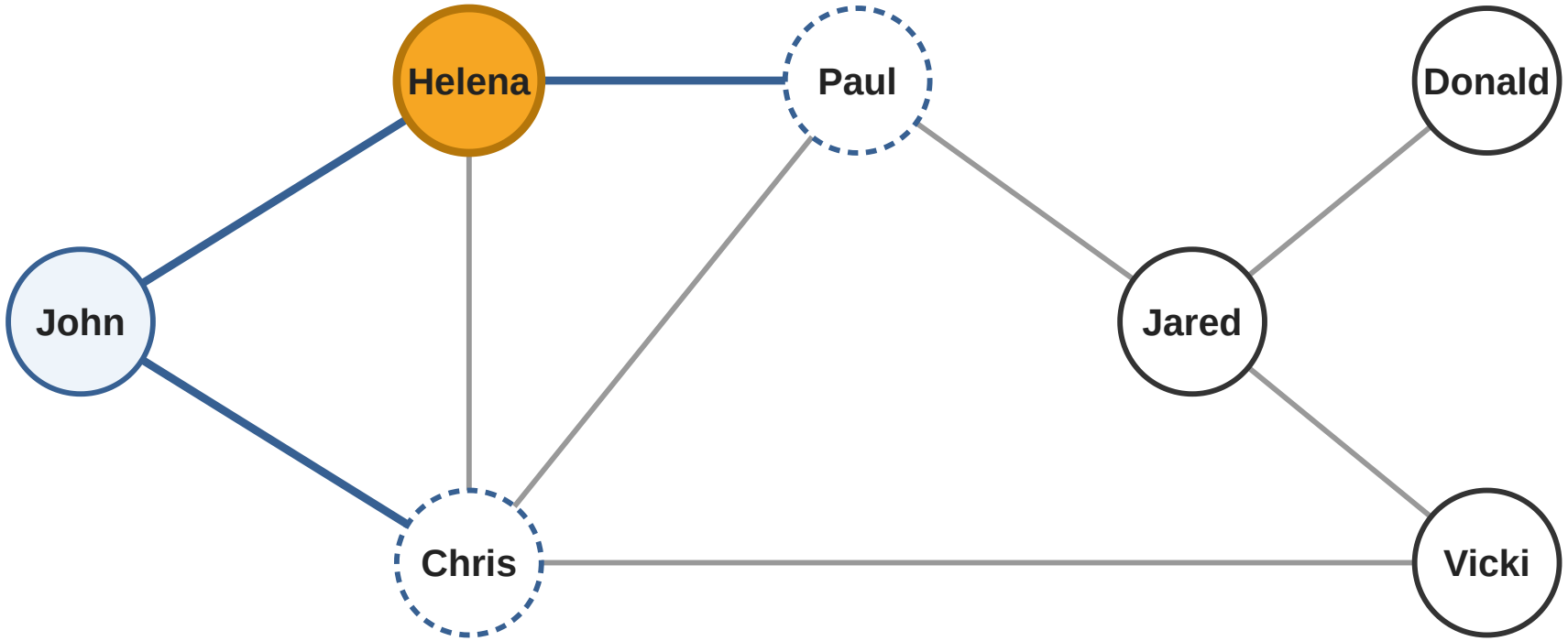
# BFS example: step 1




Pop **John** from $Q$. Discover **Helena, Chris** → add to $Q$.

$Q$ = [Helena, Chris]

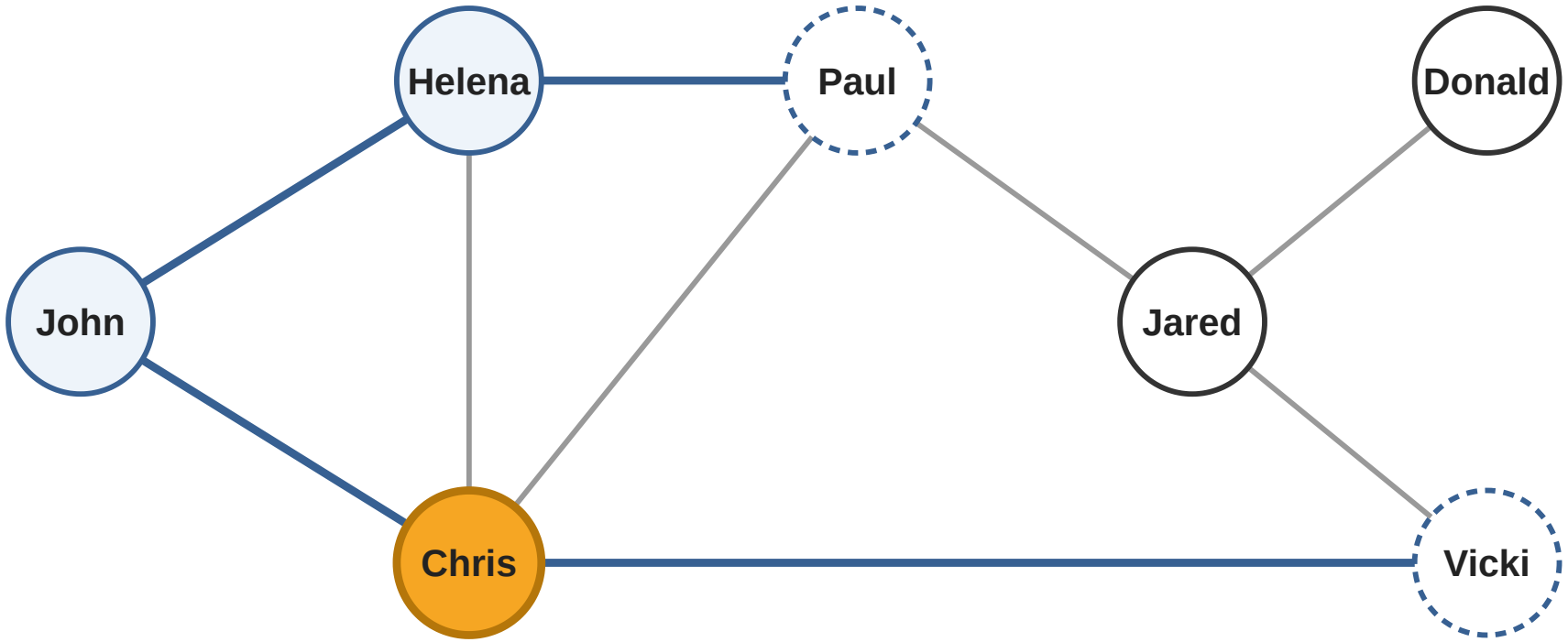
# BFS example: step 2




Pop **Helena** from $Q$. Discover **Paul** → add to $Q$.

$Q$ = [Chris, Paul]

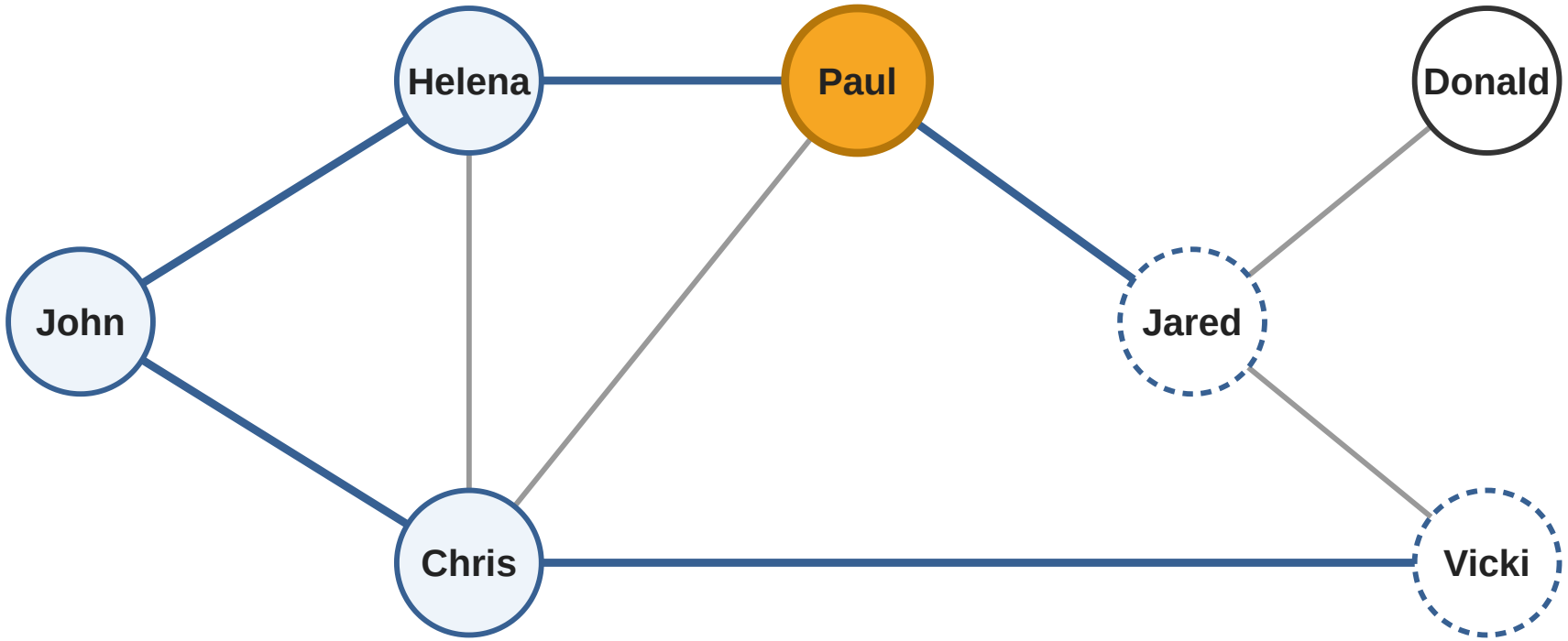
# BFS example: step 3




Pop **Chris** from $Q$. Discover **Vicki** → add to $Q$.

$Q$ = [Paul, Vicki]

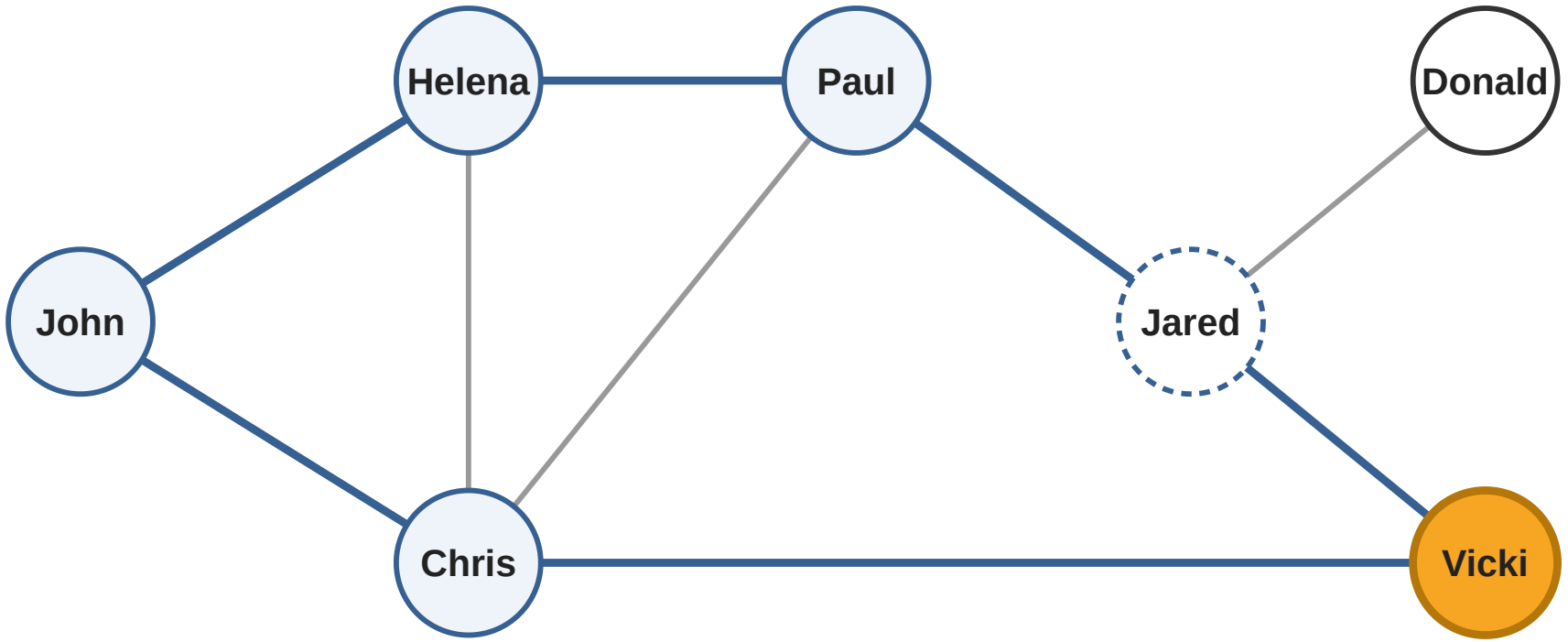
# BFS example: step 4




Pop **Paul** from $Q$. Discover **Jared** → add to $Q$.

$Q$ = [Vicki, Jared]

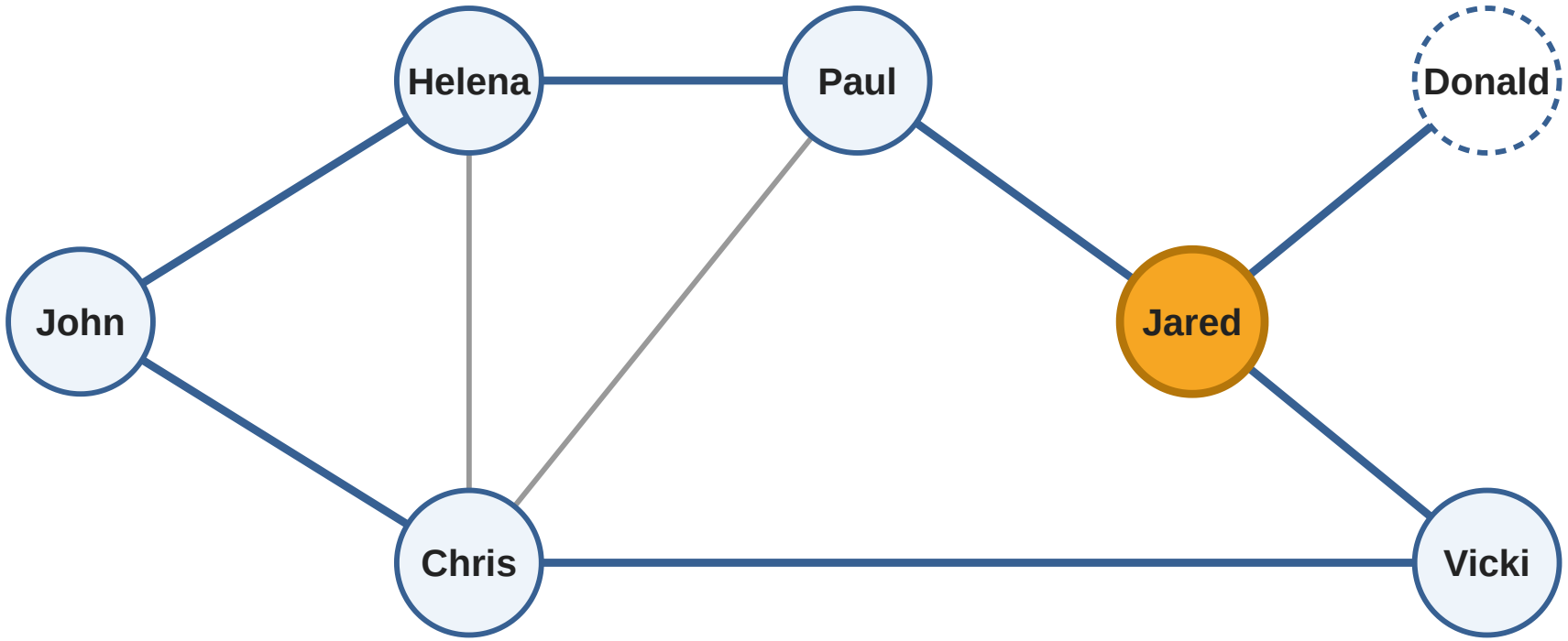
# BFS example: step 5




Pop **Vicki** from $Q$. No new nodes (Jared already queued).

$Q$ = [Jared]

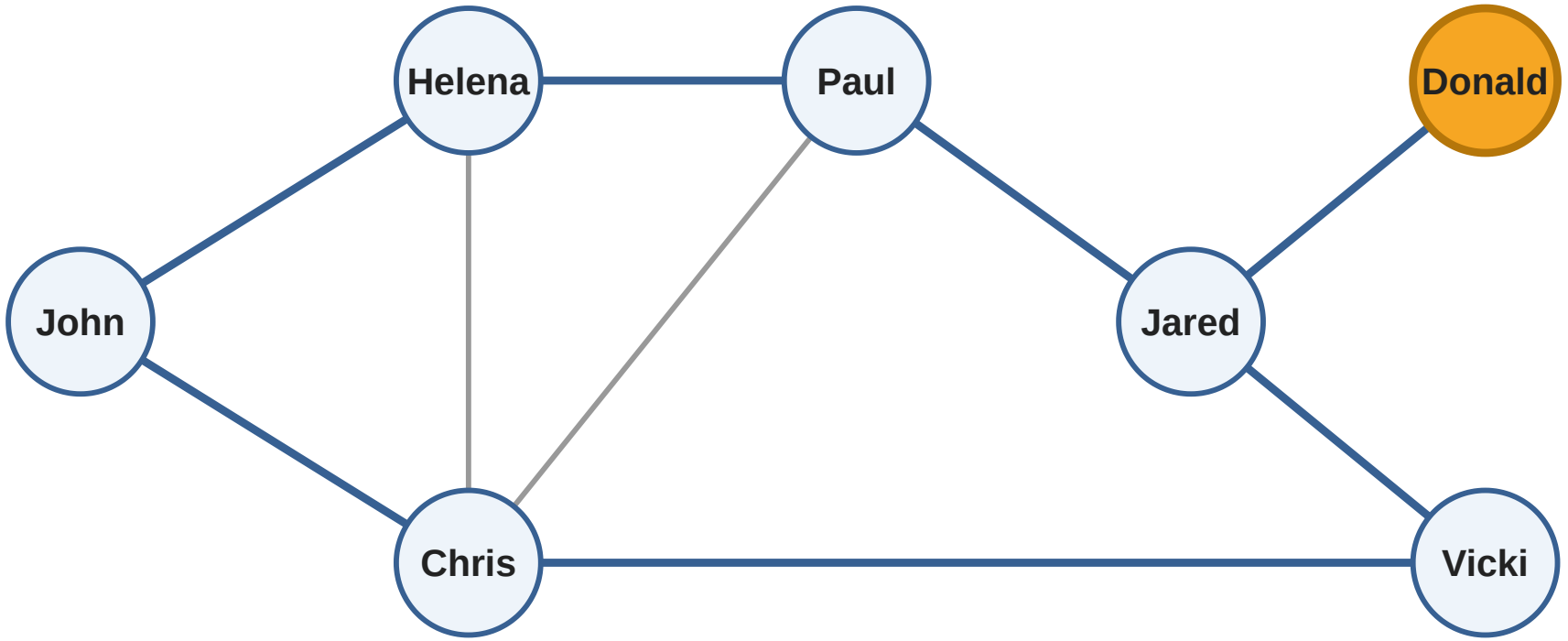
# BFS example: step 6




Pop **Jared** from $Q$. Discover **Donald** → add to $Q$.

$Q$ = [Donald]

# BFS example: step 7




Pop **Donald** from $Q$. No new nodes.

$Q$ is now empty, done!

# BFS example: full trace

| Step | Pop from $Q$ | Newly discovered | $Q$ after |
|---|---|---|---|
| 1 | John | Helena, Chris | Helena, Chris |
| 2 | Helena | Paul | Chris, Paul |
| 3 | Chris | Vicki | Paul, Vicki |
| 4 | Paul | Jared | Vicki, Jared |
| 5 | Vicki | (none) | Jared |
| 6 | Jared | Donald | Donald |
| 7 | Donald | (none) | (empty) |

# How much work is it?

**Complete exploration**: At the end, any given node $v$ is explored if and only if there is a path from $s$ to $v$

**Running time?**

Work for each of $n$ nodes:

- Enqueue: $O(1)$
- Mark explored: $O(1)$
- Dequeue: $O(1)$

Work for each of $m$ edges:

- Check if endpoint explored: $O(1)$
- Total over all edges: $O(m)$

Total: $O(n+m)$

# How to get shortest distances?

- Fewest edges to cross to get from $s$ to $v$
- This is the same as how many "layers of friends" we have to cross, or degrees of separation

To get shortest distance, we need to add:

**Initialise**: $dist(v) = 0$ for $v = s$, $\infty$ for other $v$

**Main loop**: when going through edge $(v, w)$: if $w$ unexplored, set $dist(w) = dist(v) + 1$

# We can also check if the graph is connected

**Connected components** = the "pieces" of (undirected) graph $G$

- Check if graph is disconnected
- Visualise graphs
- Clustering

**How to do it**:

- If BFS finishes without exploring all nodes, restart it from an unexplored node
- $O(m+n)$ time (the size of the entire graph)

# Why study graphs?

Many problems revolve around **transitions from one system state to another**

- Graph search often useful
- Examples: travelling, scheduling, etc.

For **unweighted graphs**, many problems solved by **BFS** or **DFS** (depth-first search):

- Linear time
- DFS explores like a maze (optional problem)
- **Weighted graphs** require different methods

# Review

Many problems can be modelled using **graphs**:

- Graph representation: adjacency list or adjacency matrix
- Graphs can be (un)directed, (un)weighted
- Breadth-first search for shortest paths in unweighted graphs

**Review exercises:**

- BFS implementation
- Optional: apply BFS to a Hollywood dataset